PAYFINIA · MONEY MOVEMENT ANALYTICS

ROI Calculator — Technical Brief

What we built, how it works, and the engineering choices behind it. USF FinTech Graduate Project.

1 · What it is

An interactive web tool that estimates how much a community bank saves by moving the payments it **sends** (checks, wires, Same-Day ACH) onto instant rails (FedNow / RTP). A banker enters their volumes and the tool returns net annual benefit, ROI, payback, and 5-year NPV — with every cost figure traceable to a public source. Two builds share identical logic: a **React** app (client-facing, deployable) and a **Streamlit** app (quick internal use).

2 · The model (how the numbers are produced)

Each rail's fully-loaded cost per transaction is built in **three layers**, so it is clear who charges what:

- **Network fee** — what the rail operator (Fed / The Clearing House) charges. Exact, published.
- **Provider fee** — what Payfinia (the TPSP) charges to host/send. Payfinia to set.
- **Internal cost** — the bank's own cost: processing/labor, failure & exception, fraud loss, fraud prevention, compliance, reconciliation, liquidity.

Total cost/txn = Network + Provider + Internal. The engine then computes:

Quantity	Formula
Savings per migrated txn	legacy rail cost – instant cost
Annual savings (per rail)	outbound volume × % shifted × savings per txn
Net annual benefit	Σ annual savings – annual platform cost
Year-1 ROI	net benefit ÷ one-time setup cost
Payback (months)	one-time cost ÷ net benefit × 12
5-year NPV	discounted net benefit over 5 yrs – one-time cost

Key result of this design: instant costs ~$0.77–0.87/txn all-in; a check ~$3 and a wire ~$19. Standard ACH is already cheaper than instant, so the model correctly shows ACH migration as net-negative — savings come from checks and wires. Only **outbound** volume is modeled because a bank only controls what it originates.

3 · Engineering choices (what we used and why)

Choice	Why
React + Vite (primary app)	Fast, interactive UI that recalculates instantly; one-command deploy to Vercel; the polished build for client demos.
Streamlit (twin app)	Zero front-end code for quick internal analysis; same model logic so results always match.
Recharts (charts)	Native, editable React charts — clean stacked/'bar' visuals without shipping images.
No backend / no database	All computation is client-side. The bank's numbers never leave their browser — privacy by default, and it deploys as a static site (cheap, secure, fast).

Single data module (data.js)	One source of truth for costs, sources and the model math. UI and the assistant both call the same functions — no drift.
Space Grotesk + Inter, tabular numerals	A deliberate fintech identity; figures align in columns like a financial statement — not a generic template.

4 · Features built for the client (and the reason)

- **Mini-calculators (Cost Builder)** — a bank can't recall '$2.00 per check', but it knows staff, salaries and volumes. We derive per-item costs from figures they actually have (e.g., staff × salary ÷ volume). Nizar's request.
- **'Use My Financials' (top-down)** — enter aggregate labor & fraud totals; the tool allocates them across rails. Grounds the estimate in the bank's real books.
- **Sanity check** — shows the implied total ops cost so it can be compared to the bank's call-report line. Justin's suggestion — instant credibility test.
- **Origination framing + monthly/annual toggle + comma formatting** — models only outbound (ACH-shrinking), and matches how banks actually report numbers.
- **Guided Savings Assistant** — a chat that asks a few questions and fills the calculator. It runs on our sourced model (not a free-form GPT), so it can never invent a statistic.

5 · Data & integrity

Every default traces to a public source — **Federal Reserve** (FedNow, Fedwire, Reg II), **Nacha** (ACH), **The Clearing House** (RTP), **AFP Payments Fraud Survey**, **FFIEC**. Each line on the Rail Data tab has a 'verify' link. Figures that are not directly citable (per-item processing, fraud prevention, compliance, reconciliation, liquidity) are labelled **Estimate** and flagged as calibration targets. Nothing is fabricated; the tool clearly separates cited facts from estimates.

6 · Run & deploy

Local: `cd 6_web_calculator && npm install && npm run dev`. **Vercel:** import the repo, set Root Directory to `6_web_calculator`, framework Vite. **Streamlit twin:** `streamlit run 3_roi_calculator/roi_calculator.py`.

This tool produces an analytical estimate, not a guarantee of savings. Cost benchmarks are conservative public-source midpoints; the model is built to be calibrated against Payfinia production data (fraud loss/item, true processing cost/item, real rail mix). USF FinTech Graduate Project · Prepared for Payfinia · 2025–2026.